Luke Johnson, Seth Grace, Thenmozhi Boopathy
Dr. Shan
ISA 632
May 15th, 2026

Data Analyst Job Finder Chatbot

Data Collection Process

The knowledge base was built from two sources. The primary source was a set of 37 curated job listing PDF files from Deloitte, JPMorgan Chase, and Amazon, stored in a Databricks Unity Catalog volume. To expand coverage across a broader range of companies and seniority levels, a secondary source was added: a CSV file containing 2,530 scraped Data Engineer roles (DataEngineer.csv), also stored in the same volume. Together, these two sources form the full knowledge base used by the chatbot.

Preprocessing

The PDF files were read using Databricks' built-in ai_parse_document() function, which extracts structured text from binary files without requiring custom parsing code. The text elements from each document were flattened and joined into a single readable block per file. The source file path was retained as a doc_uri column for traceability.

The CSV file was processed separately using a Spark transformation. The relevant fields — job title, company name, location, and job description — were concatenated into a single content string per row. Both the parsed PDF records and the transformed CSV rows were then inserted into the same Delta table, creating a unified knowledge base.

Vector Store

The preprocessed content was stored in a Delta table called jobs_knowledge_base, with Change Data Feed enabled. This table was connected to a Databricks Vector Search index (jobindex) using the databricks-gte-large-en embedding model. The vector store enables semantic similarity search, allowing the system to retrieve job listings that are conceptually relevant to a user's query even when exact keywords are not matched. The index syncs automatically whenever the Delta table is updated.

Prompt

The system prompt instructs the model to act as a job search assistant with a built-in routing layer. For job-related queries, the model must call the vector search retriever tool before responding and base its answer only on the retrieved documents. It is instructed to verify that retrieved results match the user's specified filters — such as

seniority level, job function, and location — before presenting them. If the results do not match the user's criteria, the model must use a defined MISMATCH FORMAT that clearly states what was unavailable and presents the closest available results instead. For general knowledge questions unrelated to job search, the model may respond directly without querying the index.

Model Pipeline

When a user submits a query, it is routed to the LangChain tool-calling agent deployed on a Databricks Model Serving endpoint. If the query is job-related, the agent calls the VectorSearchRetrieverTool to fetch the top semantically similar documents from the vector index. The model then reads the retrieved results and generates a structured response covering job title, company, description, qualifications, and salary where available. The underlying large language model is Databricks Llama 4 Maverick, wrapped in a LangChain agent and registered via MLflow for versioned deployment.

Evaluation

The chatbot was evaluated using MLflow's genai evaluation framework with a set of 12 realistic user questions covering role-specific searches, multi-criteria filtering, salary queries, and out-of-scope requests. Four scorers were used: Safety, Relevance to Query, and two custom Guidelines scorers — job_relevant and filter_aware.

The evaluation was run twice. The initial run, conducted on the PDF-only dataset, produced scores of 100% Safety, 92% Job Relevant, 92% Relevance to Query, and 58% Filter Aware. After the CSV dataset was added, the evaluation was re-run. Filter Aware improved from 58% to 67%, reflecting broader data coverage. However, Job Relevant dropped from 92% to 50% and Relevance to Query dropped from 92% to 67%, indicating that the less-curated CSV rows introduced retrieval noise. Safety remained at 100% across both runs. The primary lesson from evaluation is that expanding data volume without structured filtering can reduce retrieval precision.

Deployment

The agent was logged and registered to Unity Catalog via MLflow and deployed to a Databricks Model Serving endpoint using agents.deploy() with scale_to_zero enabled. A Streamlit chatbot interface was built on top of this endpoint, providing suggested prompts, session chat history, and thumbs up/down feedback on every response.

The app was deployed to two targets. First, it was deployed internally via Databricks Apps, accessible to workspace users with View permission. Second, it was deployed

publicly to Streamlit Community Cloud (analystjobfinder.streamlit.app), where it is accessible to anyone without a login. The Streamlit Cloud deployment is connected directly to the project's GitHub repository (github.com/boopatt/analystjobfinder) and redeploys automatically on every push to the main branch.

User feedback is collected through the app and stored in a chatbot_feedback Delta table via the Databricks SQL Statement Execution API. The table captures the question, response, rating, category, and optional comment for each interaction. As of the writing of this document, 18 responses have been rated — 16 positive (89%) and 2 negative — confirming that the core retrieval approach is working well for most queries.

Limitations

The most significant limitation is that the chatbot can only answer from job listings present in the knowledge base. Although the CSV expansion added 2,530 roles and improved filter-aware coverage, it also introduced retrieval noise because the CSV data is less curated than the original PDFs. This trade-off points to the need for hybrid search — combining semantic similarity with structured metadata filters on fields such as seniority level, location, and required skills.

A second limitation, identified through real user feedback, is that the system cannot answer aggregation queries such as "How many jobs are there for Deloitte?" Vector Search is a similarity retrieval tool and does not support count or aggregation operations. The planned fix is to add a SQL Statement Execution tool to the agent, enabling it to query the jobs_knowledge_base Delta table directly for structured lookups. The agent's routing prompt would classify count and aggregation queries to this SQL tool while continuing to route job search queries to the vector retriever.

A third limitation is the token output limit, which can truncate detailed responses for roles with long descriptions. Increasing the max_tokens parameter from 500 to 1,000 or more would reduce truncation without requiring any model changes.